

Android 第七章第三节教学讲义（系统交互进阶实战）

各位同学，今天咱们咱们继续学习 Android 中与系统深度交互的功能：发送彩信、查询系统图片、使用 FileProvider 共享文件以及应用安装。这些功能在社交 APP、图片管理工具、应用市场类 APP 中非常常见，实用性很强。咱们用 "场景 + 代码" 的方式，保证小白也能轻松掌握！

一、发送彩信：带图片 / 音频的消息

彩信（MMS）是可以包含图片、音频、视频的消息，比普通短信功能更丰富。发送彩信需要用到系统提供的 Intent 和相关权限。

1. 权限准备

在 AndroidManifest.xml 中声明权限：

```
<uses-permission android:name="android.permission.SEND_SMS" /> <!-- 基础短信权限 -->
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" /> <!-- 读取图片需要 -->
<uses-permission android:name="android.permission.INTERNET" /> <!-- 彩信可能需要网络 -->
```

注意：Android 6.0 + 需要动态申请 SEND_SMS 和 READ_EXTERNAL_STORAGE 权限。

2. 发送彩信（调用系统彩信界面）

推荐用这种方式，安全且无需处理复杂的彩信协议：

```
private void sendMms() {
    String phoneNumber = "13800138000"; // 收件人号码
    String textContent = "这是一条带图片的彩信！"; // 文字内容
    String imagePath = "/storage/emulated/0/Pictures/test.jpg"; // 图片路径（需存在）
    // 1. 创建意图，指定彩信动作
    Intent intent = new Intent(Intent.ACTION_SEND);
```

```
intent.setType("image/*"); // 类型：图片（彩信支持多种类型）
// 2. 设置收件人
intent.putExtra("address", phoneNumber);
// 3. 设置文字内容
intent.putExtra("sms_body", textContent);
// 4. 添加图片附件（通过Uri）
File imageFile = new File(imagePath);
if (imageFile.exists()) {
    Uri imageUri = FileProvider.getUriForFile(this, "com.example.fileprovider", imageFile);
    intent.putExtra(Intent.EXTRA_STREAM, imageUri);
    intent.addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION); // 授予读取权限
}
// 5. 启动系统彩信界面
startActivity(Intent.createChooser(intent, "发送彩信"));
}
```

说明：这里用到了FileProvider（后面会详细讲），因为 Android 7.0 + 禁止直接暴露文件路径，需要通过 Provider 共享。

二、通过 MediaStore 查询系统图片

手机中的图片（包括相机拍摄、下载的图片）都被系统的MediaStore管理，我们可以通过它查询所有图片，不用自己遍历存储卡。

1. 权限说明

- Android 10（API 29）及以上：查询图片无需权限，直接通过MediaStore访问；
- Android 9 及以下：需要READ_EXTERNAL_STORAGE权限（需动态申请）。

2. 查询所有图片（按时间排序）

```
private void queryAllImages() {
    // 1. 定义要查询的列（图片ID、路径、名称、日期）
    String[] projection = {
        MediaStore.Images.Media._ID,
        MediaStore.Images.Media.DATA, // 图片路径（Android 10+不推荐使用）
        MediaStore.Images.Media.DISPLAY_NAME, // 图片名称
    };
}
```

```

    MediaStore.Images.Media.DATE_ADDED // 添加时间（秒级时间戳）
};
// 2. 排序方式：按添加时间倒序（最新的在前）
String sortOrder = MediaStore.Images.Media.DATE_ADDED + " DESC";
// 3. 查询系统图片库
Cursor cursor = getContentResolver().query(
    MediaStore.Images.Media.EXTERNAL_CONTENT_URI, // 外部存储图片URI
    projection,
    null, // 条件
    null, // 条件参数
    sortOrder
);
// 4. 处理查询结果
if (cursor != null) {
    List<ImageInfo> imageList = new ArrayList<>();
    while (cursor.moveToNext()) {
        ImageInfo info = new ImageInfo();
        // 获取图片ID（用于构建Uri）
        long id = cursor.getLong(cursor.getColumnIndexOrThrow(MediaStore.Images.Media._ID));
        // 构建图片Uri（Android 10+推荐用这种方式访问）
        info.uri = Uri.withAppendedPath(MediaStore.Images.Media.EXTERNAL_CONTENT_URI,
String.valueOf(id));
        // 获取图片名称
        info.name = cursor.getString(cursor.getColumnIndexOrThrow(MediaStore.Images.Media
.DISPLAY_NAME));
        imageList.add(info);
    }
    cursor.close();
    // 显示图片列表（可用RecyclerView，这里简化）
    showImages(imageList);
}
}
// 图片信息实体类
class ImageInfo {
    Uri uri; // 图片Uri
    String name; // 图片名称
}
// 显示图片（示例：用ImageView显示第一张）
private void showImages(List<ImageInfo> imageList) {
    if (!imageList.isEmpty()) {

```

```
ImageView iv = findViewById(R.id.iv_image);
iv.setImageURI(imageList.get(0).uri); // 直接用Uri设置图片
}
}
```

优势：MediaStore会自动扫描所有图片，包括其他 APP 保存的图片，无需自己处理复杂的文件路径。

三、FileProvider：安全共享文件（Android 7.0 + 必备）

Android 7.0（API 24）以上禁止在 APP 间直接传递file://格式的 Uri（会抛出异常），必须用FileProvider生成content://格式的 Uri 来共享文件，更安全。

1. 配置 FileProvider 步骤

步骤 1：在 Manifest 中注册 FileProvider

```
<application ...>
  <!-- 注册FileProvider -->
  <provider
    android:name="androidx.core.content.FileProvider"
    android:authorities="com.example.fileprovider" <!-- 自定义，通常是包名+fileprovider -->
    android:exported="false" <!-- 必须为false，不允许外部直接访问 -->
    android:grantUriPermissions="true"> <!-- 允许授予临时权限 -->

    <!-- 关联路径配置文件 -->
    <meta-data
      android:name="android.support.FILE_PROVIDER_PATHS"
      android:resource="@xml/file_paths" />
  </provider>
</application>
```

步骤 2：创建路径配置文件（res/xml/file_paths.xml）

定义哪些目录的文件可以被共享：

```
<?xml version="1.0" encoding="utf-8"?>
<paths xmlns:android="http://schemas.android.com/apk/res/android">
  <!-- 共享外部存储私有目录下的图片 -->
  <external-files-path
    name="my_images" <!-- 别名，用于构建Uri -->
    path="Pictures" /> <!-- 实际路径： /Android/data/包名/files/Pictures/ -->
  <!-- 其他可选路径 -->
  <!-- <external-path name="external" path="." /> 共享整个外部存储（谨慎使用） -->
  <!-- <files-path name="internal" path="." /> 共享内部存储 -->
</paths>
```

步骤 3：使用 FileProvider 生成 Uri

```
// 要共享的文件（位于external-files-path下的Pictures目录）
File imageFile = new File(getExternalFilesDir(Environment.DIRECTORY_PICTURES), "test.jpg");
;
// 生成content://格式的Uri
Uri imageUri = FileProvider.getUriForFile(
    this,
    "com.example.fileprovider", // 和Manifest中authorities一致
    imageFile
);
```

步骤 4：授予临时访问权限

在启动其他 APP 时，通过Intent授予权限：

```
intent.addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION); // 允许读取
// intent.addFlags(Intent.FLAG_GRANT_WRITE_URI_PERMISSION); // 允许写入（如需）
```

四、应用安装：从本地安装 APK 文件

实现类似 "应用市场" 的功能，从本地存储卡安装 APK 文件（需要处理不同 Android 版本的权限和限制）。

1. 权限和配置

步骤 1：添加权限

```
<!-- 安装未知来源应用权限 (Android 8.0+) -->
<uses-permission android:name="android.permission.REQUEST_INSTALL_PACKAGES" />
<!-- 读取存储 (用于获取APK文件) -->
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE" />
```

- `REQUEST_INSTALL_PACKAGES`：无需动态申请，但需要在代码中判断是否开启；
- `READ_EXTERNAL_STORAGE`：需要动态申请（Android 6.0+）。

步骤 2：处理 Android 10 + 的文件访问

Android 10 + 推荐用 `MediaStore` 或 `FileProvider` 访问 APK 文件，这里以 `FileProvider` 为例。

2. 安装 APK 的代码实现

```
private static final int REQUEST_INSTALL_PERMISSION = 100;
// 安装APK方法
private void installApk(String apkPath) {
    File apkFile = new File(apkPath);
    if (!apkFile.exists()) {
        Toast.makeText(this, "APK文件不存在", Toast.LENGTH_SHORT).show();
        return;
    }
    // 1. 检查是否有安装未知来源应用的权限 (Android 8.0+)
    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
        boolean hasPermission = getPackageManager().canRequestPackageInstalls();
        if (!hasPermission) {
            // 没有权限，跳转到设置页开启
            Intent intent = new Intent(Settings.ACTION_MANAGE_UNKNOWN_APP_SOURCES,
                Uri.parse("package:" + getPackageName()));
            startActivityForResult(intent, REQUEST_INSTALL_PERMISSION);
            return;
        }
    }
}
```

```

}
// 2. 生成APK文件的Uri (用FileProvider)
Uri apkUri;
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.N) {
    // Android 7.0+用FileProvider
    apkUri = FileProvider.getUriForFile(this, "com.example.fileprovider", apkFile);
} else {
    // 低版本直接用file://Uri
    apkUri = Uri.fromFile(apkFile);
}
// 3. 启动安装界面
Intent installIntent = new Intent(Intent.ACTION_VIEW);
installIntent.setDataAndType(apkUri, "application/vnd.android.package-archive");
installIntent.addFlags(Intent.FLAG_ACTIVITY_NEW_TASK);
if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.N) {
    installIntent.addFlags(Intent.FLAG_GRANT_READ_URI_PERMISSION); // 授予读取权限
}
startActivity(installIntent);
}
// 处理安装权限的回调 (Android 8.0+)
@Override
protected void onActivityResult(int requestCode, int resultCode, @Nullable Intent data) {
    super.onActivityResult(requestCode, resultCode, data);
    if (requestCode == REQUEST_INSTALL_PERMISSION) {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            boolean hasPermission = getPackageManager().canRequestPackageInstalls();
            if (hasPermission) {
                // 用户开启了权限，重新执行安装
                installApk("/storage/emulated/0/Download/app.apk");
            } else {
                Toast.makeText(this, "请允许安装未知来源应用", Toast.LENGTH_SHORT).show();
            }
        }
    }
}
}
}

```

说明：APK 文件路径需替换为实际存在的路径（如下载目录的app.apk）。

总结

这节课咱们学了 4 个系统交互的进阶功能：

1. **发送彩信**：通过系统 Intent 实现，需添加图片附件并使用 FileProvider 共享路径；
2. **查询系统图片**：用MediaStore访问系统图片库，获取图片 Uri 和信息，适配不同 Android 版本；
3. **FileProvider**：Android 7.0 + 共享文件的安全方式，需在 Manifest 注册并配置共享路径；
4. **应用安装**：需处理REQUEST_INSTALL_PACKAGES权限，用 FileProvider 生成 APK 的 Uri，启动系统安装界面。

这些功能都涉及系统安全限制和版本适配，实际开发中要特别注意不同 Android 版本的差异。课后可以做一个 "图片分享 APP"，实现查询手机图片、通过彩信发送图片的功能，巩固今天的知识！有问题随时问～

（注：文档部分内容可能由 AI 生成）